Notifications — FCM, SMS & Email

App: `apps/notifications` **Channels:** Push (FCM), SMS (Twilio/MSG91), Email (SendGrid), In-App **Delivery:** All notifications are sent via Celery async tasks — never block the API response.

Table of Contents

1. Notification Architecture
2. Firebase Cloud Messaging (Push)
3. SMS via Twilio
4. Email via SendGrid
5. In-App Notification Store
6. Notification Task Reference
7. Notification Templates

---

1. Notification Architecture

```
Business event occurs (booking accepted, SOS triggered, payment
done)
         │
         ▼
Service / Signal triggers notification dispatch
         │
         ▼
Celery Task (async, high_priority queue)
         │
         ├── FCM Push → Firebase → iOS/Android device
         ├── SMS      → Twilio   → Phone number
         ├── Email    → SendGrid → Email inbox
         └── In-App   → Stored in DB → fetched on next app open
         │
         ▼
NotificationLog record (success/failure, provider_message_id)
```

Why all async via Celery? - SMS and email calls can take 500ms–2s. Blocking API responses is unacceptable. - Celery enables retries if provider is down. - NotificationLog gives full audit trail.

---

2. Firebase Cloud Messaging (Push)

```python
# apps/notifications/services/push.py

import logging
from django.conf import settings
import firebase_admin
from firebase_admin import credentials, messaging

logger = logging.getLogger(__name__)

# Initialize Firebase Admin SDK once at module load
_firebase_app = None

def get_firebase_app():
    global _firebase_app
    if _firebase_app is None:
        cred =
        credentials.Certificate(settings.FIREBASE_SERVICE_ACCOUNT_KEY_PATH)
        _firebase_app = firebase_admin.initialize_app(cred)
    return _firebase_app


class FCMService:

    @classmethod
    def send_to_device(cls, fcm_token: str, title: str, body: str,
                       data: dict = None, notification_type: str
        = '') -> dict:
        """
        Send a push notification to a single device.
        Returns {'success': True, 'message_id': '...'} or
        {'success': False, 'error': '...'}
        """
        if not fcm_token:
            return {'success': False, 'error': 'NO_FCM_TOKEN'}

        get_firebase_app()  # Ensure initialized

        message = messaging.Message(
            notification=messaging.Notification(
                title=title,
                body=body,
            ),
            data={
                'type': notification_type,
                **(data or {}),
                # All values must be strings for FCM data payload
                **{k: str(v) for k, v in (data or {}).items()},
            },
            token=fcm_token,
            android=messaging.AndroidConfig(
```

```
            priority='high',  # High priority for time-
        sensitive notifications
            notification=messaging.AndroidNotification(
                channel_id='bsecure_alerts',
                priority='high',
                sound='default',

        visibility=messaging.AndroidNotificationVisibility.PUBLIC,
            ),
        ),
        apns=messaging.APNSConfig(
            headers={'apns-priority': '10'},
        # 10 = immediate delivery (iOS)
            payload=messaging.APNSPayload(
                aps=messaging.Aps(
                    sound='default',
                    badge=1,
                    content_available=True,
                )
            ),
        ),
    )

    try:
        message_id = messaging.send(message)
        return {'success': True, 'message_id': message_id}
    except messaging.UnregisteredError:
        # Token is invalid — clear it from DB
        cls._clear_fcm_token(fcm_token)
        return {'success': False, 'error':
     'TOKEN_UNREGISTERED'}
    except Exception as e:
        logger.error(f'FCM send failed: {e}')
        return {'success': False, 'error': str(e)}

@classmethod
def send_to_multiple(cls, fcm_tokens: list, title: str, body:
     str,
                     data: dict = None) -> dict:
    """Send same notification to multiple devices (batch)."""
    if not fcm_tokens:
        return {'success_count': 0, 'failure_count': 0}

    get_firebase_app()

    messages = [
        messaging.Message(
            notification=messaging.Notification(title=title,
     body=body),
            data={k: str(v) for k, v in (data or {}).items()},
            token=token,
        )
        for token in fcm_tokens
    ]
```

```
        # FCM batch send (max 500 per batch)
        response = messaging.send_all(messages)
        return {
            'success_count': response.success_count,
            'failure_count': response.failure_count,
        }

    @classmethod
    def _clear_fcm_token(cls, invalid_token: str):
        """Remove invalid FCM token from all user profiles."""
        from apps.users.models import UserProfile
        UserProfile.objects.filter(fcm_token=invalid_token).update(fcm_token='')
```

---

3. SMS via Twilio

```
# apps/notifications/services/sms.py

import logging
from django.conf import settings
from twilio.rest import Client
from twilio.base.exceptions import TwilioRestException

logger = logging.getLogger(__name__)


class SMSService:

    _client = None

    @classmethod
    def get_client(cls) -> Client:
        if cls._client is None:
            cls._client = Client(
                settings.TWILIO_ACCOUNT_SID,
                settings.TWILIO_AUTH_TOKEN
            )
        return cls._client

    @classmethod
    def send_sms(cls, to_number: str, body: str) -> dict:
        """
        Send an SMS message.
        to_number must be in E.164 format: +919876543210
        """
        try:
            message = cls.get_client().messages.create(
                body=body,
                from_=settings.TWILIO_PHONE_NUMBER,
                to=to_number,
```

```
            )
            return {
                'success': True,
                'message_sid': message.sid,
                'status': message.status,
            }
        except TwilioRestException as e:
            logger.error(f'Twilio SMS failed to {to_number}: {e}')
            return {'success': False, 'error': str(e)}

    @classmethod
    def send_otp_sms(cls, to_number: str, otp_code: str) -> dict:
        body = (
            f'Your b-secure verification code is: {otp_code}\n'
            f'Valid for 5 minutes. Do not share with anyone.'
        )
        return cls.send_sms(to_number, body)

    @classmethod
    def send_sos_alert_sms(cls, to_number: str, user_name: str,
                           latitude: float, longitude: float) ->
        dict:
        maps_link = f'https://maps.google.com/?q={latitude},
        {longitude}'
        body = (
            f'ALERT:
        {user_name} has triggered an SOS on b-secure. '
            f'Their last known location: {maps_link}. '
            f'Please contact them immediately or call emergency
        services.'
        )
        return cls.send_sms(to_number, body)

    @classmethod
    def make_masked_call(cls, from_user_number: str,
        to_guard_number: str) -> dict:
        """
        Create a Twilio Proxy session for masked calling.
        Neither party sees the other's real number.
        """
        try:
            proxy_client = cls.get_client().proxy
            service =
        proxy_client.services(settings.TWILIO_PROXY_SERVICE_SID)
            session =
        service.sessions.create(unique_name=f'session_{from_user_number}')

        session.participants.create(identifier=from_user_number)

        session.participants.create(identifier=to_guard_number)
            return {
                'success': True,
                'session_sid': session.sid,
```

```
            'proxy_numbers': [p.proxy_identifier for p in
        session.participants.list()],
            }
        except Exception as e:
            return {'success': False, 'error': str(e)}
```

---

4. Email via SendGrid

```
# apps/notifications/services/email.py

import logging
from django.conf import settings
from sendgrid import SendGridAPIClient
from sendgrid.helpers.mail import Mail, Attachment, FileContent,
        FileName, FileType, Disposition
import base64

logger = logging.getLogger(__name__)


class EmailService:

    @classmethod
    def send_email(cls, to_email: str, subject: str,
        html_content: str,
                   attachments: list = None) -> dict:
        """Send transactional email via SendGrid."""
        if not to_email:
            return {'success': False, 'error': 'NO_EMAIL'}

        message = Mail(
            from_email=settings.DEFAULT_FROM_EMAIL,
            to_emails=to_email,
            subject=subject,
            html_content=html_content,
        )

        if attachments:
            for attachment_data in attachments:
                attachment = Attachment(

        FileContent(base64.b64encode(attachment_data['content']).decode()),
                    FileName(attachment_data['filename']),
                    FileType(attachment_data['mime_type']),
                    Disposition('attachment'),
                )
                message.attachment = attachment

        try:
            sg = SendGridAPIClient(settings.SENDGRID_API_KEY)
            response = sg.send(message)
```

```
            return {
                'success': True,
                'status_code': response.status_code,
                'message_id': response.headers.get('X-Message-
        Id'),
            }
        except Exception as e:
            logger.error(f'SendGrid email failed to {to_email}:
        {e}')
            return {'success': False, 'error': str(e)}

    @classmethod
    def send_invoice_email(cls, to_email: str, user_name: str,
                           pdf_content: bytes, invoice_number:
        str) -> dict:
        from django.template.loader import render_to_string
        html = render_to_string('notifications/emails/
        invoice.html', {
            'user_name': user_name,
            'invoice_number': invoice_number,
        })
        return cls.send_email(
            to_email=to_email,
            subject=f'Your b-secure Invoice #{invoice_number}',
            html_content=html,
            attachments=[{
                'content': pdf_content,
                'filename':
        f'bsecure_invoice_{invoice_number}.pdf',
                'mime_type': 'application/pdf',
            }]
        )
```

---

5. In-App Notification Store

```
# apps/notifications/services/inapp.py

from apps.notifications.models import NotificationLog


class InAppNotificationService:

    @classmethod
    def create(cls, user, notification_type: str, title: str,
               body: str, data: dict = None) -> NotificationLog:
        """Store an in-app notification for later retrieval."""
        return NotificationLog.objects.create(
            recipient=user,
            channel='IN_APP',
            notification_type=notification_type,
            title=title,
```

```
            body=body,
            data=data or {},
            status='SENT',
        )

    @classmethod
    def get_unread_count(cls, user) -> int:
        return NotificationLog.objects.filter(
            recipient=user,
            channel='IN_APP',
            is_read=False,
        ).count()
```

---

6. Notification Task Reference

All tasks live in `apps/notifications/tasks.py` and run on the appropriate Celery queue.

```
# apps/notifications/tasks.py

from celery import shared_task
from .services.push import FCMService
from .services.sms import SMSService
from .services.email import EmailService
from .services.inapp import InAppNotificationService
from .models import NotificationLog
import logging

logger = logging.getLogger(__name__)


def _log_result(notification_log_id, result: dict):
    status = 'SENT' if result.get('success') else 'FAILED'
    NotificationLog.objects.filter(id=notification_log_id).update(
        status=status,
        provider_message_id=result.get('message_id',
         result.get('message_sid', '')),
        failure_reason=result.get('error', ''),
    )


# ----- OTP -----

@shared_task(bind=True, max_retries=3, queue='high_priority')
def send_otp_sms(self, phone_number: str, otp_code: str):
    result = SMSService.send_otp_sms(phone_number, otp_code)
    if not result['success']:
        raise self.retry(countdown=5)
```

```python
# ----- Booking Events -----

@shared_task(queue='high_priority')
def notify_guard_assigned(booking_id: str):
    """Notify user that a guard has been assigned."""
    from apps.bookings.models import Booking
    booking = Booking.objects.select_related('user',
        'guard__user').get(id=booking_id)
    user = booking.user
    guard = booking.guard

    title = 'Guard Assigned!'
    body =
        f'{guard.user.full_name} is on the way. ETA will appear on
        your map.'
    data = {'booking_id': booking_id, 'guard_id': str(guard.id),
        'action': 'OPEN_TRACKING'}

    # Push
    if user.fcm_token:
        FCMService.send_to_device(user.fcm_token, title, body,
        data, 'GUARD_ASSIGNED')

    # In-app
    InAppNotificationService.create(user,
        'GUARD_ASSIGNED', title, body, data)

    # SMS
    SMS_body = f'b-secure: {guard.user.full_name} has been
        assigned to your booking. Track them live in the app.'
    SMSService.send_sms(user.phone_number, SMS_body)


@shared_task(queue='high_priority')
def notify_guard_arrived(booking_id: str):
    from apps.bookings.models import Booking
    booking = Booking.objects.select_related('user',
        'guard__user').get(id=booking_id)
    user = booking.user
    title = 'Guard Has Arrived'
    body = f'{booking.guard.user.full_name} is at your location.
        Please share your start OTP.'
    data = {'booking_id': booking_id, 'action': 'SHOW_START_OTP'}
    if user.fcm_token:
        FCMService.send_to_device(user.fcm_token, title, body,
        data, 'GUARD_ARRIVED')
    InAppNotificationService.create(user, 'GUARD_ARRIVED', title,
        body, data)


@shared_task(queue='high_priority')
def notify_session_started(booking_id: str):
    from apps.bookings.models import Booking
```

```
    booking =
        Booking.objects.select_related('user').get(id=booking_id)
    user = booking.user
    title = 'Session Started'
    body = f'Your security session has started. The SOS button is
        now active.'
    data = {'booking_id': booking_id, 'action': 'OPEN_SESSION'}
    if user.fcm_token:
        FCMService.send_to_device(user.fcm_token, title, body,
        data, 'SESSION_STARTED')
    InAppNotificationService.create(user, 'SESSION_STARTED',
        title, body, data)


@shared_task(queue='default')
def notify_session_completed(booking_id: str):
    from apps.bookings.models import Booking
    booking =
        Booking.objects.select_related('user').get(id=booking_id)
    user = booking.user
    title = 'Session Completed'
    body = f'Your session has ended. You were billed ₹
        {booking.total_amount}. How was your guard?'
    data = {'booking_id': booking_id, 'action': 'RATE_GUARD'}
    if user.fcm_token:
        FCMService.send_to_device(user.fcm_token, title, body,
        data, 'SESSION_COMPLETED')
    InAppNotificationService.create(user, 'SESSION_COMPLETED',
        title, body, data)


# ----- Payment Events -----

@shared_task(queue='default')
def send_wallet_topup_notification(user_id: str, amount: float):
    from apps.users.models import UserProfile
    user = UserProfile.objects.get(id=user_id)
    title = 'Wallet Topped Up'
    body = f'₹{amount:.0f} has been added to your b-secure
        wallet.'
    if user.fcm_token:
        FCMService.send_to_device(user.fcm_token, title, body,
                                  {'action': 'OPEN_WALLET'},
        'WALLET_TOPUP')
    InAppNotificationService.create(user, 'WALLET_TOPUP', title,
        body)


@shared_task(queue='default')
def send_payment_receipt(booking_id: str):
    from apps.bookings.models import Booking
    booking =
        Booking.objects.select_related('user').get(id=booking_id)
    user = booking.user
```

```
    title = 'Payment Receipt'
    body = f'₹{booking.total_amount} charged for your session.
        Invoice will be emailed shortly.'
    if user.fcm_token:
        FCMService.send_to_device(user.fcm_token, title, body,
                                  {'booking_id': booking_id,
        'action': 'OPEN_INVOICE'},
                                  'PAYMENT_RECEIPT')
    InAppNotificationService.create(user, 'PAYMENT_RECEIPT',
        title, body)


@shared_task(queue='low_priority')
def send_invoice_email(booking_id: str, s3_key: str):
    """Fetch invoice PDF from S3 and email to user."""
    from apps.bookings.models import Booking
    import boto3
    from django.conf import settings
    import io

    booking =
        Booking.objects.select_related('user').get(id=booking_id)
    user = booking.user

    if not user.email:
        return  # No email on file

    s3 = boto3.client('s3')
    pdf_obj =
        s3.get_object(Bucket=settings.AWS_STORAGE_BUCKET_NAME,
        Key=s3_key)
    pdf_content = pdf_obj['Body'].read()

    invoice_number = f'BSE-{str(booking.id)[:8].upper()}'
    EmailService.send_invoice_email(user.email, user.full_name,
        pdf_content, invoice_number)


# ----- SOS Events -----

@shared_task(bind=True, max_retries=5, queue='high_priority')
def notify_emergency_contacts(self, sos_alert_id: str):
    """Send SMS to all emergency contacts when SOS is
        triggered."""
    from apps.sos.models import SOSAlert, EmergencyContactAlert
    from apps.users.models import EmergencyContact

    sos =
        SOSAlert.objects.select_related('user').get(id=sos_alert_id)
    contacts = EmergencyContact.objects.filter(user=sos.user)

    for contact in contacts:
        result = SMSService.send_sos_alert_sms(
            to_number=contact.phone_number,
```

```
            user_name=sos.user.full_name,
            latitude=float(sos.latitude),
            longitude=float(sos.longitude),
        )
        EmergencyContactAlert.objects.create(
            sos_alert=sos,
            contact_name=contact.name,
            contact_phone=contact.phone_number,
            sms_sent=result['success'],
        )

    if not contacts.exists():
        logger.warning(f'SOS
        {sos_alert_id}: user has no emergency contacts!')


@shared_task(queue='high_priority')
def notify_guard_of_user_sos(booking_id: str):
    """Notify guard if their active client triggers SOS."""
    from apps.bookings.models import Booking
    booking =
        Booking.objects.select_related('guard__user').get(id=booking_id)
    guard_user = booking.guard.user
    title = '  Client SOS Alert'
    body =
        'Your client has triggered an emergency SOS. Stay alert
        and check on them immediately.'
    if guard_user.fcm_token:
        FCMService.send_to_device(guard_user.fcm_token, title,
        body,
                                  {'booking_id': booking_id,
        'action': 'SOS_ALERT'},
                                  'CLIENT_SOS')


# ----- Guard Document Events -----

@shared_task(queue='default')
def notify_document_approved(guard_id: str, document_type: str):
    from apps.guards.models import GuardProfile
    guard =
        GuardProfile.objects.select_related('user').get(id=guard_id)
    user = guard.user
    title = 'Document Approved'
    body = f'Your {document_type.replace("_", " ").title()} has
        been verified.'
    if user.fcm_token:
        FCMService.send_to_device(user.fcm_token, title, body,
        {}, 'DOC_APPROVED')
    InAppNotificationService.create(user, 'DOC_APPROVED', title,
        body)


@shared_task(queue='default')
```

```
def notify_document_rejected(guard_id: str, document_type: str,
        reason: str):
    from apps.guards.models import GuardProfile
    guard =
        GuardProfile.objects.select_related('user').get(id=guard_id)
    user = guard.user
    title = 'Document Needs Resubmission'
    body = f'Your {document_type.replace("_", " ").title()} was
        rejected. Reason: {reason}'
    if user.fcm_token:
        FCMService.send_to_device(user.fcm_token, title, body,
        {}, 'DOC_REJECTED')
    SMSService.send_sms(user.phone_number, f'b-secure: {body}')
    InAppNotificationService.create(user, 'DOC_REJECTED', title,
        body)
```

---

7. Notification Templates

Push Notification Types Reference

notification_type	Title	Body	data.action
GUARD_ASSIGNED	Guard Assigned!	{name} is on the way	OPEN_TRACKING
GUARD_ARRIVED	Guard Has Arrived	Share your start OTP	SHOW_START_OTP
SESSION_STARTED	Session Started	SOS button is now active	OPEN_SESSION
SESSION_COMPLETED	Session Completed	Billed ₹{amount}	RATE_GUARD
PAYMENT_RECEIPT	Payment Receipt	₹{amount} charged	OPEN_INVOICE
WALLET_TOPUP	Wallet Topped Up	₹{amount} added	OPEN_WALLET
SOS_ACKNOWLEDGED	SOS Acknowledged	Control room is responding	OPEN_SOS
DOC_APPROVED	Document Approved	{type} verified	OPEN_DOCS
DOC_REJECTED	Document Rejected	Resubmission needed	OPEN_DOCS
BOOKING_CANCELLED	Booking Cancelled	{reason}	OPEN_BOOKINGS
CHECKIN_MISSED			OPEN_SESSION

notification_type	Title	Body	data.action
	Guard Check-in Missed	Contact guard or report	
NEW_BOOKING_REQUEST	New Booking!	{service} — {distance}km	OPEN_REQUEST
PAYOUT_PROCESSED	Payout Sent	₹{amount} transferred	OPEN_EARNINGS
CLIENT_SOS	Client SOS Alert	Check on your client	OPEN_SESSION